Yelp Recommendation System

Distributed Data Systems

Sai Mulagan

March 13, 2026

1. Abstract

This project outlines an end-to-end recommendation system build using the Yelp Academic Dataset. We designed an automated pipeline that pulls up to 100,000 raw Yelp reviews from Google Cloud Storage, orchestrates data processing through Apache Airflow, and manages the data in MongoDB Atlas. We used these extra datasets to train a Funk SVD collaborative filtering model, and compared analytical query performance between MongoDB aggregations and Apache Spark. Also, we containerized our recommendation engine with Docker and deployed it via a FastAPI on Google Cloud Run. By precomputing and storing the top 10 recommendations per user, the system ensures low-latency API responses. Ultimately this project demonstrates how to effectively integrate cloud storage, workflow orchestration, NoSQL databases, machine learning, and containerization with Docker into a distributed, functional and productional data system.

2. Introduction

First of all, we chose the Yelp Academic Dataset because it naturally fits the core objectives of a distributed data system. It provides large-scale, semi-structured JSON data files with real-world user-item interactions, which allows us to tackle both data engineering and data science tasks simultaneously. On the data engineering side, metadata like timestamps and engagement(e.g. useful, funny, cool) are ideal for testing complex MongoDB aggregations and Spark DataFrames. On the data science side, the fundamental variables like user_id, business_id and stars are exactly what is needed to train a collaborative filtering model using Funk SVD.

Our project was developed in 3 phases over a six-week timeline. Initially, the scope was purely analytical: loading raw data into Google Cloud Storage, migrating it to MongoDB Atlas via Apache Airflow, running SparkSQL queries, and training a matrix factorization model to predict user ratings.

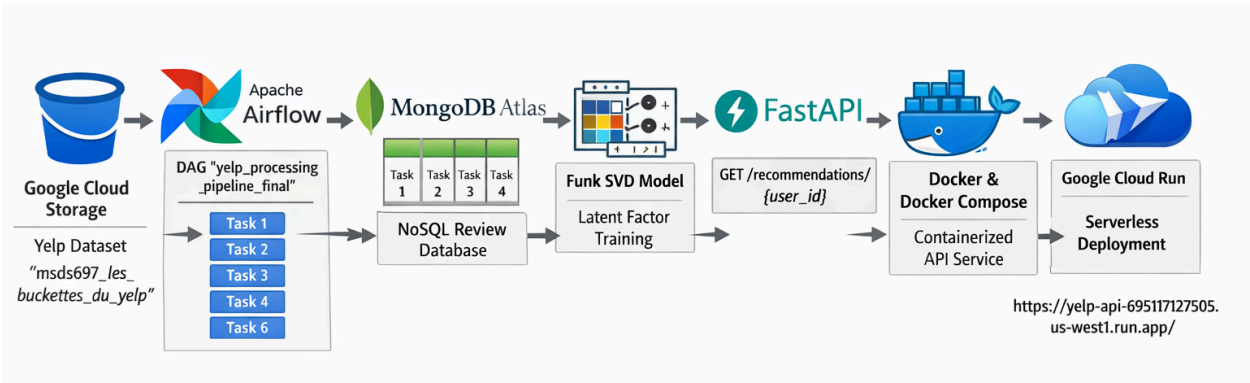
However, as development progressed, we decided to expand the architecture to simulate a real-world production environment. Rather than stopping at offline model training, we fully automated the pipeline to precompute the top 10 recommendations for each user and store them back in MongoDB for fast retrieval. To complete the system, we built a FastAPI application to serve these recommendations, containerized it with Docker, and deployed it to Google Cloud Run. The final result is a complete pipeline that transitions smoothly from raw data ingestion to a live, scalable application.

3. Data Story

The Yelp datasets available here offered several different files including a file of business attributes, a file of reviews, and a file of user_attributes. The review file was of special interest because it includes user_id, business_id, and review score. The machine ML component of this assignment coincided with the Funk SVD sparse matrix concept were were learning in Advanced Machine learning at the time we reviewing candidate data sets for the project. The canonical Funk SVD application requires a set containing user_id, business_id, and review scores, and seeks to predict the ratings that a user *would* make for a movie that he or she has not seen. –This is the ML mechanism powering movie recommendation systems on sites like Netflix. Interestingly, these three elements are all that are required –the user and

business attributes that would usually be explicitly required in a predictive model are *implicitly* conveyed by the user_id or business_id to which they are tied. The additional tables containing these attributes are then required only for secondary reporting purposes – e.g. what sort of business was predicted to well (or poorly) for what sort of business.

4. System Architecture



The system architecture is straightforward: Because the data for this ML exercise was static and relatively small, we were able to upload it GCS from the terminal with basic command lines.

Apache Airflow, executing DAGs, was then used to: (1) move the data into the MongoDB Atlas storage, and make basic tables that would accommodate our analysis. (2) execute the FunkSVD model training, and store the model's recommendations at the per-usr_id level. (3) create auxiliary reporting tables. (4) Fast API was used to expose the user_id-level data retrieval function. (5) Google Cloud Run hosted the Fast API Service. (6) Docker was used to containerize the API service.

With all this in place, we had a service which could quickly retrieve the likely scores for a user even for business they had not visited. The business application would be to show the user those business having the highest ratings (the ones they're most likely to enjoy). Of course, this data can be further joined to attributes for filtering, or for photos for the user's UI experience.

5. Storage

To begin, our raw JSON files were directly uploaded to a Google Cloud Storage bucket. Through an Airflow DAG, the data was directly transferred to the cloud NoSQL database, MongoDB Atlas. We chose this for it's ability to hold semi-structured JSON data such as our review data, along with its ability to build powerful aggregation pipelines. Atlas served as the primary data store, holding our four collections:

- *raw_reviews* - This is our original Yelp review JSON loaded directly.
- *yelp_businesses* - This is our original Yelp business JSON loaded directly.

- *business_star_stats* - This is the aggregation of statistics per business (average rating, review count, engagement metrics).
- *yearly_star_stats* - This collection holds aggregated statistics per business *per year*.
- *recommendations* - This was the precomputed top-10 recommended businesses for each user. Computing the model in real time for every request would be far too intensive and computationally expensive. We opted to store the computed recommendations in MongoDB Atlas, allowing the API to grab results quickly and efficiently.
- *Location-limitedrecommendations* - This collection holds the filtered version of the recommendations, including for each user only those recommendations for establishments in their estimated "hometown."

6. Data Processing

The data processing pipeline is managed using Apache Airflow. The pipeline processes Yelp review data stored in Google Cloud Storage and loads it into MongoDB. Each step in the pipeline prepares the data for the next step.

The first task loads Yelp review data into the MongoDB raw_reviews collection. For testing and development, only the first 5000 reviews are loaded. This keeps the pipeline fast and makes it easier to test the system.

After the raw data is loaded, MongoDB aggregation pipelines are used to create new collections with summary information. The create_business_star_stats task groups reviews by business_id and calculates statistics such as average rating and review count. The create_yearly_star_stats task extracts the year from the review date and calculates yearly average ratings and review counts for each business.

Next, the pipeline prepares the training data for the recommendation model. The export_training_data task extracts the user_id, business_id, and stars fields from MongoDB and saves them as a dataset for training. This dataset represents the ratings that users gave to different businesses.

The train_recommender_model task trains a recommendation model using the Funk SVD algorithm. The model learns patterns in the rating data and tries to understand which businesses a user may like.

Finally, the generate_and_store_recommendations task uses the trained model to predict ratings for different users and businesses. The system selects the highest predicted scores and stores the top recommended businesses for each user in the MongoDB recommendations collection. These recommendations are later returned by the API.

As an extra flourish, we added a rudimentary system to narrow these recommendations down to the user's "hometown." Since we did not have access to actual location data, we implemented a rough estimation by simply using MongoPy queries to check which towns they reviewed in, and assuming their most-reviewed town to be their "hometown," then returning only those recommendations for establishments located in that town.

7. Results and Performance Evaluation

The recommendation system is able to generate personalized business recommendations using Yelp review data. The Funk SVD model is trained using the exported user and business rating dataset. The model was evaluated using Root Mean Squared Error (RMSE).

During testing, the model achieved an RMSE of about 1.33. This shows that the model can reasonably predict user ratings based on past review data. Because only the first 5000 Yelp reviews were used, the dataset is small compared to the full Yelp dataset. Using more data could improve the model performance.

The system also performs well when serving recommendations through the API. Instead of generating recommendations each time a request is made, the system generates recommendations ahead of time and stores them in MongoDB.

When a user sends a request to the FastAPI service, the API simply looks up the stored recommendations in the database. This makes the API very fast because it does not need to run the machine learning model during each request.

The deployed API on Google Cloud Run can return results quickly while also scaling automatically if more requests are made.

8. Distributed Systems Considerations

Our primary distributed data system was MongoDB Atlas. As we learned in class, this is a fully-managed cloud-based NoSQL database running on a distributed cluster with built in replication. I would like to say that it was chosen because of the structure of our data being semi-structured JSONs, but the reality is that this was a requirement of the project! It just happened to be the perfect fit for this type of data anyways, win-win! Another reason Atlas was perfect for us is that the built in aggregation pipelines allowed us to computer our business statistics with no client required, which simplified our project. Should we have wanted to host the entire massive dataset, we would have been able to pay to do so!

Airflow is what we used to orchestrate our distributed workflow. Airflow ran our 6-step Distributed Acyclic Graph (DAG) containing explicit dependencies. This was perfect for us because our project had clear sequential order with dependencies (e.g., export of training data must happen after aggregation).

It is also worth mentioning Spark and Google Cloud Storage. Spark was useful for loading data into Spark dataframes and querying with SQL to compare performance of queries. Google Cloud Storage allowed for our raw data to be held online in a bucket. This ensured that data storage and processing were kept separate.

9. Challenges and Solutions

There were several challenges while building this recommendation system. One challenge was managing Python environments and library compatibility. The recommendation model originally

depended on libraries that did not work well with Python 3.13. This caused installation errors when setting up the environment. The problem was solved by switching the project environment to Python 3.11 and using one requirements file that works with Docker.

Faulty airflow UI created a large headache in trying to figure out if the airflow document was performing as expected. Repeated failed runs with no system logs would occur because of improperly downloaded/set-up software. Long debugging times and frustrating circular conversations with AI led to failures. The fix would make most IT Technicians laugh as after hours, deleting Airflow and reinstalling it let me know that it was working perfectly all along.

We learned that streaming JSON ingestion is essential for large dataset, but bulk loading caused memory issues. Our fix was to simply work with a smaller slice of the dataset to save money on computing as we did not want to pay for the Atlas cluster. Time and compute was not the worry, simply how much it would cost to host the data for a school project.

A design challenge was deciding how the API should generate recommendations. Running the machine learning model every time a user requests recommendations would be slow and expensive. Instead, the system generates recommendations in advance during the pipeline stage and stores them in MongoDB.

While implementing the filter to narrow recommendations down to those in the user's (estimated) hometown, in addition to the obvious challenge discussed above of lacking actual location data, we also found that choosing the most common entry of a subarray is not as simple in MongoPy as it would be with a Pandas dataframe, requiring substantial wrangling of GroupBy clauses to achieve without (computationally expensive) unwinding. Frankly, in a real-world environment it might simply be better to use pandas operations for a similar purpose, but given the focus of this course we opted for a Mongo solution.

Another challenge was that our workflow process was slightly out of order, as we added Atlas as a data storage facility after creating most of the pipeline. This caused the need to manually change all the inputs in all the files where a local Mongo URI was called and to switch it into a variable called in a environment variable to keep account logins private. After much trial and error, this was implemented well though! In the future, we will be doing all data setup before further pipeline work!

Sparsity of data in the yelp system created hardships in accuracy of the recommendation quality. Other issues that impacted this included Funk SVD model tuning such as learning rate and regularization.

10. Conclusion

Despite these challenges, in the end we successfully built an end-to-end automated data pipeline from raw Yelp reviews to a deployed recommendation API, with use of multiple distributed systems integrated into it: GCS, MongoDB Atlas, Apache Airflow, Spark, Docker, and Google Cloud Run. We were able to extract users' reviews, create a model of the various users' preferences, generate recommendations based on those modeled preferences, and filter those recommendations based on the user's estimated hometown, achieving reasonable recommendation quality. The project reinforced the

importance of pipeline automation, data quality checks, and system integration in real-world data engineering.

If we were to continue this project in the future, we could increase its real-world applicability by scaling up to the full Yelp dataset, adding real-time recommendation updates, and (if we could get access to such data) incorporating actual geodata for the user's location instead of estimating their location. We could also increase the range of data used for our model, and thus the quality of our recommendations, by further business metadata for hybrid recommendations, and incorporating non-numeric used feedback information such as Yelp "check-ins" on top of the numeric review data we currently use.